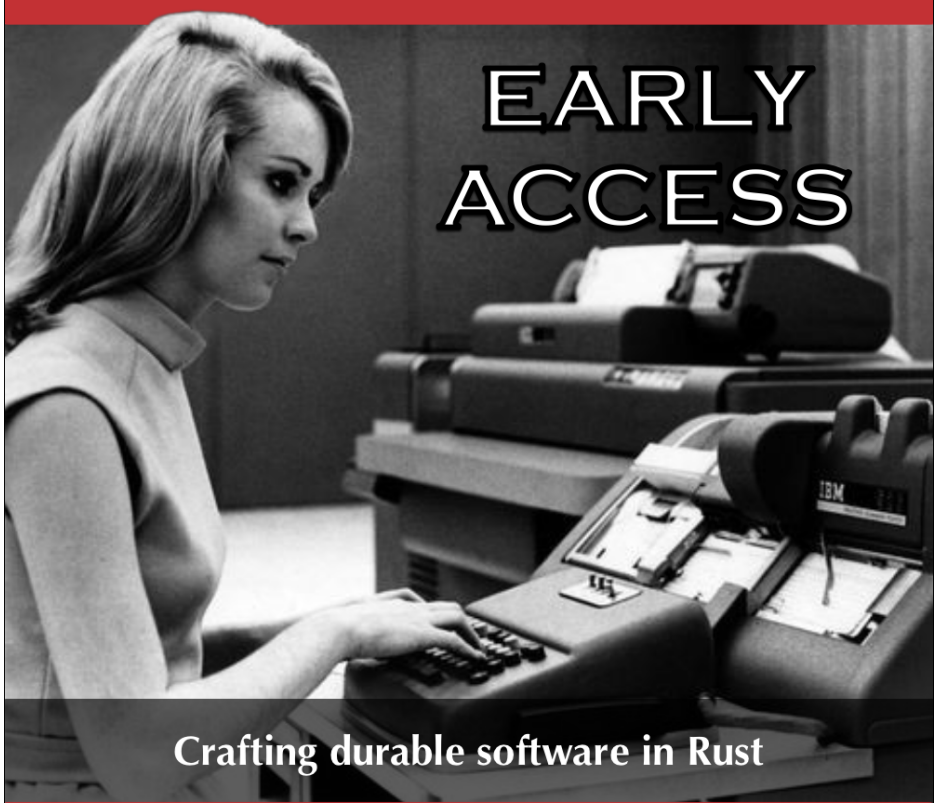
THE SECRETS OF RUST TOOLS

"Opens up a whole new way of programming"
—Jawahir Sheikh



Crafting durable software in Rust

JOHN ARUNDEL

Contents

Praise for 'The Secrets of Rust: Tools' **10**

Introduction **11**
Who is this book for? . . . 11
What should I know before reading it? . . . 12
How can I play along at home? . . . 12
How do I install Rust? . . . 12
How do I complete the challenges? . . . 13
Where do I find the listings and example solutions? . . . 13
What you'll learn . . . 13

1. Crates **14**
The universal library . . . 14
Crates are a force multiplier . . . 15
The standard library . . . 15
Raiders of the lost crate . . . 16
The de facto standard library . . . 17
Trusted, stable components . . . 17
Writing libraries, not programs . . . 18
Hello as a service . . . 18
The simplest thing imaginable . . . 18
Durable software . . . 19
Why most software sucks . . . 20
The Rust advantage . . . 20
Durability starts with testing . . . 21
Testing also drives better design . . . 21
Testing . . . 21
Rust's built-in test framework . . . 21
Failing tests . . . 22
A practice test . . . 23
Verifying the test by "bebugging" . . . 23
Writing helpful failure messages . . . 24
Test-driving our design . . . 24
Testing a function that prints . . . 24
Injecting a writer . . . 25
`Stdout` is a writer . . . 26
Writing to a `Vec` for testing . . . 26
Is this the right API? . . . 27
Don't print, return data instead . . . 27

A data-oriented `hello` crate . . . 27
Modifying `world` to return a string . . . 28
Over to you . . . 28
Variations on a theme . . . 29
Checking the test . . . 29
Test names should be sentences . . . 30
Tests as docs . . . 30
The real program . . . 31
Design and testability . . . 31

2. Paperwork **32**
Counting lines . . . 33
A prototype line counter . . . 33
Counting lines . . . 33
What do users want? . . . 34
The "magic function" approach . . . 34
Testing `count_lines` . . . 35
Taking a `BufRead` parameter . . . 36
Returning a `usize` . . . 36
Passing the test . . . 37
Moving tests into a module . . . 38
Anatomy of a test module . . . 38
Errors . . . 39
Results . . . 39
Readers are fallible . . . 39
How `lines` handles errors . . . 40
Does our program handle errors? . . . 40
Endless iteration . . . 41
Errors on standard input . . . 41
Handling errors . . . 42
Fixing a bug with a test . . . 42
A fallible reader . . . 42
Returning a `Result` . . . 43
Handling errors in `count_lines` . . . 44
The iterator version . . . 44
Displaying results to users . . . 45
A slightly nicer presentation . . . 45
Standard error, and exit status . . . 46
Resilience matters . . . 47
Handling invalid input . . . 48

3. Arguments **49**
Taking a filename . . . 50
Getting command-line arguments . . . 50
What even are arguments? . . . 50
Extracting the value . . . 50

Handling the “no path” case . . . 51
Usage conventions . . . 52
Opening the file . . . 52
Ownership what now? . . . 52
Handling the “can’t open” case . . . 53
A rough first draft . . . 54
Throwing the paperwork away . . . 55
Returning a `Result` from `main` . . . 55
Turning `None` into `Err` . . . 56
The user’s view of errors . . . 56
Not all errors are `io::Error` . . . 57
Wanted: an “any old error” type . . . 57
Returning `anyhow::Result` . . . 58
Adding context to errors . . . 58
Taking multiple arguments . . . 59
A simple prototype . . . 59
Adding context to errors . . . 60
The `skip` method . . . 60
A problem of ownership . . . 61
Challenge of a lifetime . . . 62
Making copies with `clone` . . . 62
Adding more context . . . 63
Unnecessary cloning . . . 64
Lazy closures and `with_context` . . . 65
Dumb insolence . . . 66
The magic variable . . . 66
A higher-level abstraction . . . 68
Composing abstractions . . . 69

4. Flags **71**
Counting words . . . 71
Flags . . . 72
A test for a word-counting function . . . 72
Implementing `count_words` . . . 73
An efficient line reader . . . 74
Using `read_line` for word counting . . . 74
Updating `count_lines` . . . 75
Duplication versus complication . . . 76
Preparing for the refactoring . . . 76
A struct to the rescue . . . 77
Initialising the `Count` struct . . . 78
The refactored `count` function . . . 78
Cleaning up . . . 79
Testing commands . . . 80
Taking a flag . . . 80
Integration tests . . . 81

Our first integration test . . . 81
Running the program from a test . . . 82
Introducing `assert_cmd` . . . 82
`cargo_bin` and getting the binary name . . . 83
Asserting exit status . . . 84
Asserting specific output . . . 84
Avoiding fragile tests . . . 84
Predicates . . . 85
Testing the test . . . 85
A little bebugging . . . 85
Testing the happy path . . . 86
Comparing output with expectations . . . 87
A test for the word-counting flag . . . 88
Implementing the flag . . . 89
The flag-handling logic . . . 90
Running the word counter . . . 91
Using `clap` . . . 92
More complications . . . 92
Flags and arguments can be a lot . . . 92
Introducing `clap` . . . 93
Crate features . . . 93
Deriving an argument parser . . . 93
Adding `clap` metadata . . . 94
Using the parser . . . 94
Testing the parser . . . 95
The `--help` flag . . . 96
Documenting arguments . . . 96
The heavy lifting . . . 97
`clap` is magic . . . 98
You don't need `clap`, unless you do . . . 98

5. Files **99**
A logbook in Rust . . . 99
A first experiment . . . 100
A quick reminder . . . 100
Importing traits by name . . . 100
When files don't exist . . . 101
Creating the file . . . 101
File descriptors . . . 102
File modes . . . 102
Specifying "open options" for files . . . 103
Taking a message on the command line . . . 103
Joining space-separated arguments . . . 104
Reading the logbook . . . 104
Adding a user interface . . . 105
Reading an empty logbook . . . 105

The prototype logbook . . . 106
Building a logbook library . . . 107
Reading the logbook . . . 108
Returning an `Option` . . . 108
`match` versus `if let Some` . . . 109
Appending a message . . . 109
Testing `read` . . . 109
Mismatched types . . . 110
Strings and strings . . . 111
Slicing strings . . . 111
Implementing the “no file” behaviour . . . 111
Implementing the “empty file” behaviour . . . 112
Implementing the “read file” behaviour . . . 113
Designing the “append” API . . . 114
A first test for `append` . . . 115
Testing smarter, not harder . . . 115
Checking assumptions . . . 115
A feeble test . . . 116
What we’re really testing . . . 116
Cleanup and paperwork . . . 116
A self-cleaning temporary directory . . . 117
Holding the wormhole open . . . 118
More string theory . . . 118
Introducing `Path` and `PathBuf` . . . 119
The `AsRef<T>` trait . . . 119
Taking `AsRef<Path>` in `append` . . . 119
Taking `AsRef<Path>` in `read` . . . 120
Implementing `append` . . . 120
Hello, Clippy . . . 121
Testing the “append to file” behaviour . . . 122
Bebugging `append` . . . 123
The magic `main` . . . 123
A published crate . . . 124
The README file . . . 124
The crate documentation . . . 125
Inner doc comments . . . 125
Generating the docs . . . 125
Outer doc comments . . . 126
Exploring the docs . . . 127
Intra-doc links . . . 128
Documentation is design . . . 128
Crate metadata . . . 129
Updating `Cargo.toml` . . . 130
Publishing to `crates.io` . . . 131
Using the tool . . . 132
Updating the crate . . . 132

6. Data **133**
A persistent `Vec` . . . 133
Remember the milk . . . 134
A quick sketch . . . 134
Designing `open` with a test . . . 135
Validating the test . . . 136
The "no data" case . . . 136
Implementing `open` . . . 137
Collecting an iterator of `Result` . . . 138
Writing `sync` . . . 138
A round-trip test . . . 139
`sync`: a first try . . . 140
A shorter version . . . 141
Achievement unlocked: memos . . . 141
Creating a type . . . 141
A more useful abstraction . . . 141
Adding behaviour to a `Vec` . . . 142
Structs and `Sized` fields . . . 142
Adapting the tests: `open` . . . 143
Adapting the tests: `sync` . . . 144
Refactoring `open` and `sync` for the new type . . . 145
Inherent implementations . . . 146
`Path` to `PathBuf`: the `From` trait . . . 146
`as_ref`: From `AsRef` to reference . . . 147
Building the data `Vec` . . . 147
`sync`: the making of a method . . . 148
A quick reality check . . . 149
Extending the abstraction . . . 149
The definition of "done" . . . 149
Designing a `Memo` struct . . . 150
Defining a new `enum` type . . . 150
Serialization: from data to bytes . . . 151
Storing data as JSON . . . 151
Autogenerating serialization code with Serde . . . 151
Deriving `Serialize` / `Deserialize` . . . 152
Updating the tests: `open` . . . 153
Implementing `PartialEq` . . . 154
Deriving `Debug` . . . 155
Updating the tests: `sync` . . . 155
Implementing `sync` using `serde_json` . . . 156
Refactoring `open` to read JSON . . . 157
Implementing `Display` for our types . . . 158
Creating a new `Memo` . . . 159
Constructing some test data . . . 159
Using `sync` and `open` to test each other . . . 160
Extending the user interface . . . 161

A done flag . . . 161
Don't make users do paperwork . . . 161
Matching memos by substring . . . 162
Adding a done flag . . . 162
The magic `find_all` function . . . 163
Testing `find_all` . . . 164
Implementing `find_all` . . . 165
The user experience . . . 166
Purging completed memos . . . 166
The `--purge` flag in action . . . 169
Playing well with others . . . 169

7. Clients **170**
A weather client . . . 170
The Weatherstack API . . . 171
Making HTTP requests with `reqwest` . . . 171
A first sketch . . . 171
The API response . . . 172
From sketch to crate . . . 173
Just like that: a magic `get_weather` function . . . 174
Testing `get_weather` . . . 174
A failing implementation . . . 175
Designing for testability . . . 176
A problem of determinism . . . 176
What can we test about `get_weather`? . . . 177
Eating the elephant one bite at a time . . . 177
Testing `request` . . . 178
Implementing `request` . . . 179
Blocking or non-blocking? . . . 179
Extracting weather data from the response . . . 180
Deserializing the JSON . . . 181
A surfeit of structs . . . 181
Using JSON Pointer . . . 182
Implementing `deserialize` . . . 183
Taking it for a trial run . . . 184
A nicer `Display` . . . 184
Mixing arguments and environment variables . . . 185
Mock testing . . . 186
What could be wrong? . . . 187
Why don't we want to make API calls in tests? . . . 187
A fake server using `httpmock` . . . 188
Kicking the tyres . . . 188
Configuring routes on the mock server . . . 189
Mocking the real API . . . 189
Injecting the base URL . . . 190
A provider abstraction . . . 191

Building the `Weatherstack` provider . . . 191
Compulsory return values with `must_use` . . . 192
`get_weather` becomes a method . . . 192
Putting `Weatherstack` to work . . . 193
Bebugging the test . . . 194
Checking the mock's assertions . . . 194
Putting it all together . . . 195
Using mocks with care . . . 196
Integration testing . . . 197
Designing test-friendly APIs . . . 197
Adding a "Fahrenheit" feature . . . 197
A ruthlessly minimal solution . . . 198
An `into_fahrenheit` method . . . 199
Dealing with quantities . . . 200
The newtype pattern and tuple structs . . . 200
Making `Weather` unit-agnostic . . . 201
A `Temperature` struct with internal units . . . 201
Converting between units . . . 202
Testing conversions . . . 202
Handle units and quantities with care . . . 203
Formatting temperatures for display . . . 203
Drawing the line . . . 204

8. Commands **205**

There's more to come! **206**

About this book **207**
Acknowledgements . . . 207
About me . . . 207
Feedback . . . 207
Free updates to future editions . . . 208
Join my Code Club . . . 208
Code For Your Life . . . 208
For the Love of Go . . . 209
Further reading . . . 209

Praise for 'The Secrets of Rust: Tools'

I've tried to learn Rust before, but bounced off it somehow. This book unlocked something for me, and now I have a better understanding of just what makes Rust so different.
—Lawrence Denning

I can't praise this book enough. It's opened my eyes to a whole new way of programming.
—Jawahir Sheikh

I really enjoy the project-based style—it helps to clarify how Rust's features actually come together into real programs.
—Nick Chandler

Gentle, funny, and full of clear explanations—one of the best introductory Rust books I've read.
—Flavio Balioni

It seems a little paid-by-the-word—the writer kind of goes around in circles, philosophizing and pontificating and asking rhetorical questions a bit too much.
—Hacker News commenter

JARED: *Richard, adversity is a great teacher. Just like cigarette burns.*
—"Silicon Valley"